

1. Write a C program to implement Merge sort algorithm for sorting a list of integers in ascending order.

```
int a[20];
void MergeSort(int,int);
void Merge(int,int,int);
void main()
{
    int i,n;
    printf("\n Enter array size:");
    scanf("%d",&n);
    printf("\nEnter array elements:");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    MergeSort(0,n-1);
    printf("\nArray elements after sorting:");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
}
void MergeSort(int low, int high)
{
    int mid;
    if (low < high)
    {
        mid = (low + high)/2;
        MergeSort(low, mid);
        MergeSort(mid+1, high);
        Merge(low, mid, high);
    }
}
void Merge(int low, int mid, int high)
{
    int h, i, j, k, b[50];
    h = low;
    j = mid+1;
    i = low;
    while ((h <= mid) && (j <= high) )
    {
        if( a[h] <= a[j] )
        {
            b[ i ] = a[ h ];
            h = h+1;
        }
        else
```

```

        {
            b[ i ] = a[ j ];
            j = j+1;
        }
        i = i+1;
    }
    if( h > mid )
        for ( k = j ; k <= high; k++ )
        {
            b[ i ] = a[k]; i = i+1;
        }
    else
        for ( k = h ; k <= mid; k++ )
        {
            b[ i ] = a[ k ]; i = i+1;
        }
    for( k = low; k <= high; k++ ) // copy all the elements back to array A.
        a[ k ] = b[ k ] ;
}

```

2. Write a C program to implement Quick Sort algorithm for sorting a list of integers in ascending order.

```

int a[20];
void QuickSort(int, int);
int Partition(int, int);
void Interchange(int, int);
void main()
{
    int i,n;
    printf("\n Enter array size:");
    scanf("%d",&n);
    printf("\nEnter array elements:");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    QuickSort(0,n-1);
    printf("\nArray elements after sorting:");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
}
void QuickSort(int low, int high)
{
    int j;
    if (low<high)

```

```

    {
        j = Partition(low, high);
        QuickSort(low,j-1);
        QuickSort(j+1,high);
    }
}

int Partition(int low, int high)
{
    int pivot, i, j;
    pivot = a[low];
    i =low;
    j= high+1;
    while(i < j)
    {
        i++;
        while(a[ i ] < pivot)    // while(a[ i ] < pivot && i<=high) use it
            when given array elements
                i++;            // are in decreasing order
        j--;
        while(a[ j ] > pivot)    // while(a[ j ] > pivot && j>=low) use it
            when given array elements
                j--;            // are in decreasing order
        if(i < j)
            Interchange(i, j);
    }
    Interchange(low, j);
    return j;
}

void Interchange (int x,int y)
{
    int temp;
    temp=a[x];
    a[x]=a[y];
    a[y]=temp;
}

```